Engine Trials

Action 10A / 10 points / Game Development Pipeline



Current phase / between supported and independent

1. Instructor-led practice: The instructor demonstrates and guides the complete process.

2. Supported practice: You apply the process while the instructor provides checkpoints and feedback.

3. Independent practice: The goal ahead: you direct your planning, research, building and testing using your teams and tools.

We are not fully independent yet. Responsibility is growing, with instructor lectures, checkpoints and feedback still guiding the work. Fully independent work comes later, using your teams and tools.

Choose an engine for a small game idea. Keep your existing story or user need. Today you investigate the tools and test one visible change. A finished game is not required.

1 Recap — 2 Compare — 3 Test — 4 Explain — 5 Submit

Start with last week on the whiteboard

With your group, trace input -> parse/validate -> record objects and list -> display/filter -> save. Identify one class, variable, method and test. Photograph the board and save a sentence about a peer correction.

One submission, before class ends

GitHub URL in Canvas. Receipt inside GitHub. Keep one README with class notes, comparison, test result and OOP map; add the board and trial screenshots. Reuse class evidence. The receipt records your declarations, not attendance, correctness or a grade.

00 / Pre-start: open your GitHub course repository, create Week10/Action10A on an approved data drive, confirm commit/push, and read R1-R5. Use prepared tools; avoid new purchases and long installations. Canvas supplies the official deadline.

Lecture / keep the class on task

Between supported and independent: We are between supported practice and independent practice, with growing responsibility for our own work. We are not fully independent yet. The instructor continues to guide the class through lectures, checkpoints and feedback. Use the requirements, documentation, your team, AI and other tools to choose a next step, test it and ask focused questions. Fully independent work comes later; it still uses teams and tools rather than requiring you to work alone.

Suggested pacing: 20 min recap / 20 min lecture / 25 min research / 50 min trial / 25 min OOP, AI and peer check / 20 min handoff. Adjust for the scheduled break.

Recap first

Point from JSON/text input to validation, record objects, a list, the UI and saved output. Ask: which method changed the data?

Engine, editor, game

The engine supplies runtime systems. The editor arranges a project. A game is the project you make and run. raylib supplies code functions rather than a full editor.

Three hats

Visual: camera, scene and feedback. Logical: input, conditions and methods. Data: position, score and state. Use one small object to connect them.

Supporting technology

SDK/compiler builds C#. Runtime executes it. GPU/graphics API draws. Physics tests collisions. Assets supply images, models and audio. Git records source changes.

OOP in a scene

A class describes a kind; an object is one instance. Fields/properties hold state; methods do work. Components add responsibilities; inheritance specializes a type. Engine APIs differ.

From events to a loop

WPF reacts to clicks. A game repeatedly reads input, updates state and draws a frame. Delta time helps movement track elapsed time rather than frame count.

Access and trade-offs

Compare open-source, proprietary and source-available terms separately from 2D/3D and C# support. Enterprise use is a scale/support context, not a mutually exclusive engine type. Portable does not guarantee no SDK or GPU requirements.

CLI and tooling

CLI means command-line interface. Unity editor batch-mode flags automate builds/tests; the CLI is not a separate engine. Use documented commands and existing access. Automation and AI do not replace checking the result.

Close the class

Peer-check the visible change and OOP map. Commit evidence and receipt; submit one GitHub URL. A discovery trial is enough today; a finished game is not required.

Engine comparison

Read three rows. Follow official links. Choose one tool to test. C# tools are recommended, not mandatory. Source check: October 25, 2026.

Tool / category	Code / terms	Setup and use
Godot .NET Full engine / 2D / 3D Official site / Guide	C# or GDScript Open source / MIT	Extract the official .NET editor. C# also needs a compatible .NET SDK; standard/store editions do not include C#. Good first C# editor trial
Stride 4.4 Full engine / 2D / 3D Official site / Guide	C# Open source / MIT	Use the instructor-approved 4.4 build or official setup. The class portable package is separate from the official installer; school-PC compatibility must be checked. C# scenes, entities and components
Prowl Full engine / 3D Official site / Guide	C# Open source / MIT	Upstream lists 1.0 Preview 5, .NET 10 and Vulkan 1.2. Expect changing APIs; record the version and test class hardware before choosing it for a project. C# exploration candidate / preview
Unity Full engine / 2D / 3D Official site / Guide	C# Commercial/proprietary / terms apply	Use an approved installed editor. Hub, licensing and downloads can take time; no purchase is needed for this investigation. Established C# editor workflow
Flax Full engine / 3D Official site / Guide	C#, C++ or visual scripting Source available / custom terms	Official launcher/install workflow. Source availability alone does not mean an MIT-style open-source license. Compare mixed-language scripting

Compare different dimensions

A full engine includes runtime systems and usually an editor. A library gives code building blocks. Open source, proprietary and source available describe terms; 2D/3D describes use. Enterprise describes scale or support. Record these separately.

Engine comparison / continued

Read three rows. Follow official links. Choose one tool to test. C# tools are recommended, not mandatory. Source check: October 25, 2026.

Tool / category	Code / terms	Setup and use
raylib + raylib-cs Code library / 2D / 3D Official site / Guide	C library; C# through raylib-cs Open source / zlib	No full scene editor. raylib-cs uses a .NET project and native libraries; follow its binding instructions, not an editor tutorial. See the game loop directly in code
Unreal Engine Full engine / real-time 3D Official site / Guide	C++ and Blueprints Commercial/proprietary / terms apply	Large editor/download and hardware needs. C# is not the built-in gameplay scripting route; source access is not the same as an open-source license. Compare visual scripting and high-end tools
GameMaker Specialist engine / 2D Official site / Guide	GML Code and GML Visual Commercial/proprietary / terms apply	Use existing access or investigate the official tutorial. GML is its own language, not C#. Compare event-driven 2D authoring
RPG Maker MZ Specialist engine / 2D RPG Official site / Guide	Events; JavaScript plugins Commercial/proprietary / terms apply	Identify the edition. MZ is not C#; RPG Maker Unite is a separate Unity-based product. A paid license is not required for this Action. Compare story/map/event tools

Unity CLI is tooling

A command-line interface can ask an installed editor to run a build or test. Unity editor batch-mode arguments are not another game engine. No CLI installation is required today.

Official Unity command-line reference

Record your decision

Tool 1 / category / C# support / setup / official link
Tool 2 / category / C# support / setup / official link
Tool 3 / category / C# support / setup / official link
My idea and user: ... . I choose ... because ... .

Test one visible change

1. Open your approved engine or an official example. Record its name and version.

2. Capture the starting object, label or sprite. Predict one change in text, color, size or position.

3. Change that property. Run or preview. Capture the result.

4. Write expected -> observed -> next step. Save the editable trial source and its credits.

Godot beginner route / editor-only

Use an approved editor. The .NET edition and a compatible SDK are needed only if you add C#. This small editor trial can use the standard edition.

1. In Project Manager, choose Create. Name the project EngineTrial and use an approved data-drive folder. Use the instructor-approved renderer.

2. Create a 2D Scene (Node2D), then add a child Label. Select the Label; Inspector contains its editable properties.

3. Set Text to Beacon offline. Place the label inside the visible game area. Save the scene as EngineTrial.tscn and capture the editor.

4. Change Text to Beacon online. Run the current scene with F6 or the editor command. Confirm the new text in the game window; capture and save.

Official nodes/scenes guide

Another engine or a blocked setup

Use that engine's chart link for an equivalent one-property test. A raylib-cs trial changes its basic example and reruns it. If setup blocks progress, ask the instructor to approve a working partner/instructor demonstration or official interactive example. Make your own prediction, observation and notes; record the block and approval honestly. Do not claim you ran software you only watched.

OOP / point to the object

TYPE: Beacon	INSTANCE: Dock A	INSTANCE: Dock B
Name / Online / Activate()	Online: True	Online: False

Activate() changes the first object. The second object keeps its own state.
Class: The type description. Beacon describes the kind.

Instance: One object made from a class. Each instance has its own state.

Field/property: A named data value, such as Online or Name.

Method: A named action. Activate changes the current object.

Composition / component: Combine parts with separate jobs, such as drawing and collision.

Inheritance: Specialize a base type. A scene parent-child relationship alone does not prove C# inheritance.

Trace this ordinary C# example

Read together or run in a new C# Console App, replacing Program.cs. This is not an engine script. Predict which object changes.

```
// Concept example only: ordinary C#, not an engine script.
var first = new Beacon { Name = "Dock A" };
var second = new Beacon { Name = "Dock B" };
first.Activate();
Console.WriteLine(first.Online);  // True
Console.WriteLine(second.Online); // False

class Beacon
{
    public string Name { get; set; } = "";
    public bool Online { get; private set; }
    public void Activate() { Online = true; }
}
```

Visual hat: feedback and appearance. **Logical hat:** input and action. **Data hat:** stored state and ownership. Game loop: input -> update -> draw -> repeat.

Optional C# / required AI check

Godot .NET bridge: the label changes itself

Optional extension, not an extra requirement. Select the Label and attach a C# script named StatusLabel.cs. Replace that file with this code. Use Godot .NET and the SDK listed in its official guide; build and run the scene. Do not paste this script into another engine.

```
using Godot;

public partial class StatusLabel : Label
{
    private int _checks = 0;

    public override void _Ready()
    {
        _checks++;
        Text = $"Checks: {_checks}";
    }
}
```

: Label specializes Label. _checks stores an integer. _Ready is a callback. Text is the display property. Expected output: Checks: 1.

Official Godot C# setup and API guide

Ask AI one focused question

I am comparing game engines for a beginner C# class. My tool and version are [name/version]. My smallest test is [one visible change]. Explain one relevant class/object/component relationship in plain language. Give an official documentation link. Mark any uncertain API names. I will check the link and compare the explanation with my test; do not generate a whole game.

No live AI? Review the supplied response

Supplied AI-style response for review: A class describes a kind of object; an instance has its own state. A component adds a responsibility to an object. Different engines name these differently: Unity uses GameObjects/components; Godot uses nodes/scenes. A scene hierarchy alone does not prove C# inheritance. Check the selected engine's official scripting reference before using an API name. Decision to make: keep the general analogy, correct it for your engine, and cite the source you checked.

Keep: the question, one answer point, the official link you checked, and your keep/change/reject decision. A private chat export or paid AI account is not required.

Check requirements / submit once

R1 / Recap on the whiteboard / 2 points

With your group, trace last week: input -> parse/validate -> objects/list -> display/filter -> save. Label one class, variable, method and test. Keep the board photo and a short peer-check note.

R2 / Compare three tools / 2 points

In README.md, give each tool's category, C# support, setup need and official link. Choose one to test and explain why it fits your game idea or user. C# tools are recommended; another tool is allowed.

R3 / Test one small change / 3 points

Open one engine/library example, identify its version, change one visible property and run or preview it. Save before/after evidence and a one-line expected -> observed result. Keep your editable trial source; use the instructor-approved alternative if setup blocks you.

R4 / Explain the objects / 2 points

Map one game object to a class/type, two data values and one method/action. Explain a component or inheritance relationship and where input -> update -> draw belongs. Save one AI question, an official-source check and your keep/change/reject decision; supplied-response fallback is available.

R5 / Push and submit the receipt / 1 point

Save README.md, evidence, the editable trial source and downloaded 10A receipt in GitHub. Commit, push, test instructor access, then submit the GitHub repository URL to Canvas before class ends.

Receipt, in four moves

1. In the online Action's Submit step, enter your name, GitHub URL and tested tool. Select the honest test mode and check only the requirements you have completed.

2. Download cs077-10a-receipt.txt. Save it in Week10/Action10A/Evidence/. Keep README, trial source, board and before/after evidence with it.

3. Commit and push. Open the GitHub URL and verify instructor access. Keep engine binaries, caches and private account data out of Git.

4. Return to the matching Canvas Action and submit the GitHub repository URL before class ends. Check Canvas confirms it. A downloaded receipt alone is not submission.

Reuse class evidence. Each learner submits their own link and identifies their contribution. Contact the instructor for approved attendance/access alternatives. Canvas is the official record; this document does not set a new deadline or verify attendance.